NANYANG TECHNOLOGICAL UNIVERSITY

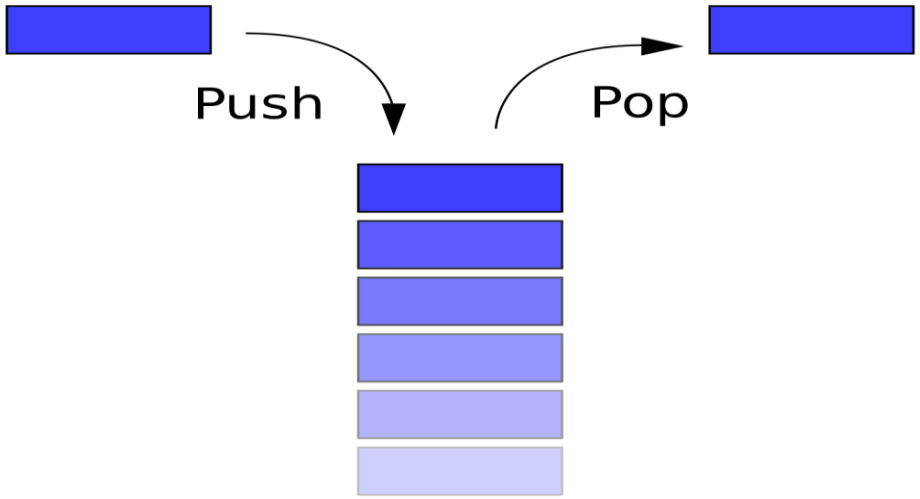
Data Structures & Algorithms in Python: Stacks

Dr. Owen Noel Newton Fernando

College of Computing and Data Science

What is a stack?

- A *stack* is a Last In, First Out (LIFO) data structure
- Anything added to the stack goes on the "top" of the stack
- Anything removed from the stack is taken from the "top" of the stack
- Things are removed in the reverse order from that in which they were inserted



Content Copyright Nanyang Technological University

Some uses of stacks

Stacks are used for:

- Any sort of nesting (such as tracking parentheses)
- Evaluating arithmetic expressions (and other sorts of expressions)
- Implementing function or method calls (built-in to virtually every programming language)
- Keeping track of choices yet to be made, or steps yet to be done.
- Keeping track of previous choices (as in search backtracking algorithms)

Content Copyright Nanyang Technological University

Example: Using a stack to reverse a sentence

```
s.push("Hello")
s.push("World")
s.push("It's")
s.push("Me")
print(s.pop(), end=" ")
print(s.pop(), end=" ")
print(s.pop(), end=" ")
print(s.pop())
```

Stack:

Me
It's
World
Hello

Output:

Content Copyright Nanyang Technological University

Implementing a Stack class using a built-in Python list

```
class Stack(object):
    def __init__(self, max):                # Constructor
        self.__stackList = [None] * max     # The stack stored as a list
        self.__top = -1                     # No items initially

    def push(self, item):                       # Insert item at top of stack
        self.__top += 1                         # Advance the pointer
        self.__stackList[self.__top] = item     # Store item

    def pop(self):                              # Remove top item from stack
        top = self.__stackList[self.__top]      # Top item
        self.__top -= 1                         # Decrease the top pointer
        return top                              # Return top item

    def peek(self):                               # Return top item
        if not self.isEmpty():                    # If stack is not empty
            return self.__stackList[self.__top]   # Return the top item

    def isEmpty(self):                      # Check if stack is empty
        return self.__top < 0
```

Content Copyright Nanyang Technological University

Implementing a Stack class using a built-in Python list

```
class Stack(object):
    def __init__(self, max):                # Constructor
        self.__stackList = [None] * max     # The stack stored as a list
        self.__top = -1                     # No items initially
```

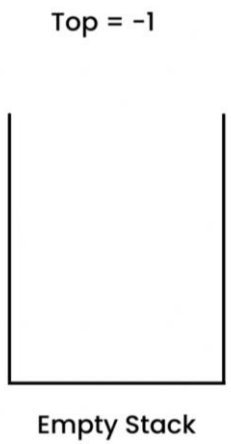
Constructor (__init__ method):

The **__init__** method initializes the stack:

Parameters: max defines the maximum number of items that can be held in the stack.

Attributes:

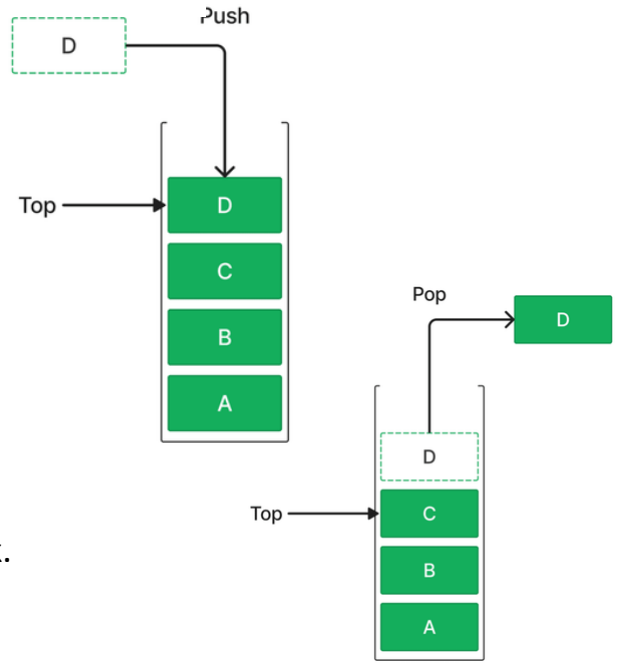
- **self.__stacklist**: A list with a fixed size (max) to store stack items. It starts with all **None** values to represent empty slots.
- **self.__top**: A pointer representing the index of the topmost item in the stack. Initially, it's set to **-1**, indicating an empty stack.



Content Copyright Nanyang Technological University

Implementing a Stack class using a built-in Python list

```
def push(self, item):                          # Insert item at top of stack
    self.__top += 1                            # Advance the pointer
    self.__stackList[self.__top] = item        # Store item

def pop(self):                                 # Remove top item from stack
    top = self.__stackList[self.__top]         # Top item
    self.__top -= 1                            # Decrease the top pointer
    return top                                 # Return top item
```



push method

Adds an item to the top of the stack.

Process:

First, **self.__top** is **incremented** by **1** to point to the next empty slot.

Then, item is placed at the position indicated by **self.__top** in self.__stacklist.

pop method

Removes and returns the item at the top of the stack.

Process:

top is set to the value at **the self.__top index**.

The self.__top pointer is then **decremented** by **1** to move down one level in the stack.

The method returns the item that was removed.

Content Copyright Nanyang Technological University

Implementing a Stack class using a built-in Python list

```
def peek(self):                              # Return top item
    if not self.isEmpty():                   # If stack is not empty
        return self.__stackList[self.__top] # Return the top item

def isEmpty(self):                       # Check if stack is empty
    return self.__top < 0
```

peek method

Returns the item at the top of the stack **without removing it.**

Process:

First, it checks if the stack is not empty using the isEmpty method.
If not empty, it returns the item at the index **self.__top.**

isEmpty method

Checks if the stack is empty.

Process:

Returns True if self.__top is less than **0**, meaning no items are in the stack;
otherwise, it returns **False.**

Content Copyright Nanyang Technological University

Critique of Stack

- Error checking: can use of Stack ever result in a **run-time exception**?
- Are there other bugs in this implementation?

Content Copyright Nanyang Technological University

Critique of Stack

Popping an Empty Stack (Stack Underflow):

Trying to pop an item from an empty stack would cause an underflow.

```
if self.__top < 0:
    raise Exception("Stack underflow")
```

The updated code snippet checks if **self.__top < 0** in the **pop()** method. If self.__top is less than 0, it means the stack is empty.

Raising an **Exception** with a message like "**Stack underflow**" prevents the method from attempting to access an invalid index, avoiding a runtime error.

Content Copyright Nanyang Technological University

Critique of Stack

Pushing to a Full Stack (Stack Overflow):

Trying to push an item to a Full stack would cause an overflow.

```
if self.__top == len(self.__stackList) - 1:
    raise Exception("Stack overflow")
```

The updated push() method checks if **self.__top** is equal to **len(self.__stackList) - 1**, which would indicate the stack is already at maximum capacity.

If this condition is true, an **Exception** with a message like "**Stack overflow**" is raised. This prevents pushing more items than the stack can hold and avoids an IndexError.

Content Copyright Nanyang Technological University

Critique of Stack

Interfering with Garbage Collection (gc):

```
self.__stackList[self.__top] = None
```

When an item is popped, it's essential to set **self.__stackList[self.__top]** to None.

This clears the reference to the removed item, allowing Python's garbage collector (gc) to **free the memory** associated with the removed object, especially for complex data types or objects.

By setting the popped position to None, you **avoid potential memory leaks**, which could happen if references are retained.

Content Copyright Nanyang Technological University

Application: Using a stack to make sure that brackets and parens are balanced

- ([ ] ({ ( ) } [( )])) is balanced;
  ([ ] ({ ( ) } [( ))]) is not

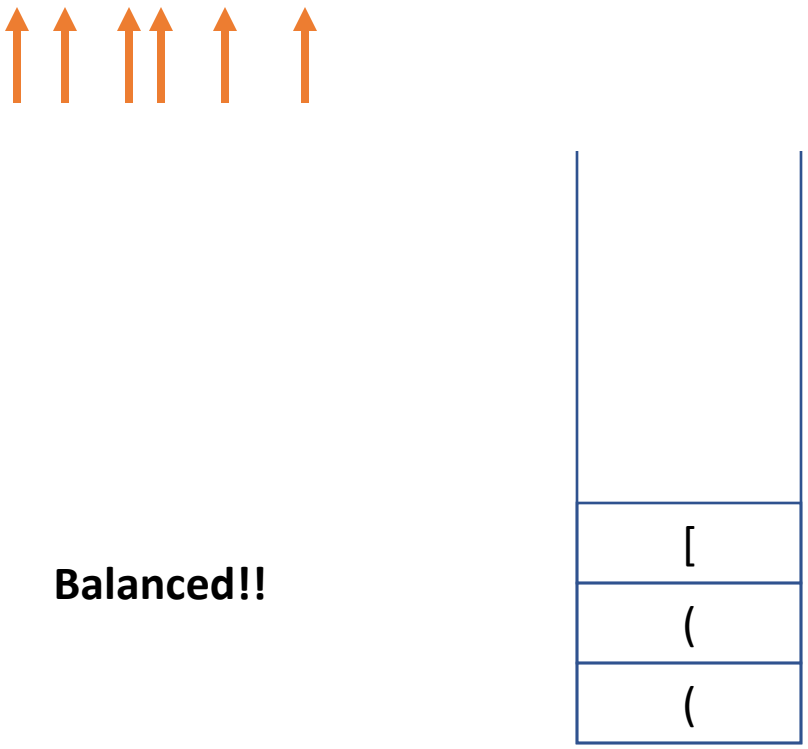
- Simple counting is not enough to check the balance

- You can do it with a stack: going left to right,
  - If you see a **(**, **[**, or **{**, **push** it on the stack
  - If you see a **)**, **]**, or **}**, **pop** the stack and check whether you popped the corresponding (, [, or {
  - When you reach the end, check that the **stack is empty**

Content Copyright Nanyang Technological University

Application: Using a stack to make sure that brackets and parens are balanced (animation)

(([]))



Balanced!!

Content Copyright Nanyang Technological University

Bracket checker using a Stack

```
from SimpleStack import *
stack = Stack(100)                    # Create a stack to hold delimiter tokens
expr = input("Expression to check: ")
errors = 0                            # Assume no errors in expression
for pos, letter in enumerate(expr): # Loop over letters in expression
   if letter in "{[(":                # Look for starting delimiter
      stack.push(letter)              # Put left delimiter on stack
   elif letter in "}])":              # Look for closing delimiter
      if stack.isEmpty():             # If stack is empty, no left delimiter
         print("Error:", letter, "at position", pos)
         errors += 1
      else:
         left = stack.pop()         # Get left delimiter from stack
         if not (left == "{" and letter == "}" or  # Do delimiters
                 left == "[" and letter == "]" or  # match?
                 left == "(" and letter == ")"):
            print("Error:", letter, "at position", pos)
            errors += 1

# After going through entire expression, check if stack empty
if stack.isEmpty() and errors == 0:
   print("Delimiters balance in expression", expr)
elif not stack.isEmpty(): # Any delimiters on stack weren't matched
   print("Expression missing right delimiters for", stack)
```

Content Copyright Nanyang Technological University

Complexity of bracket checker?

- What is the Big-O complexity of the bracket checker?
- In other words, as the size of the string being checked by the bracket checker grows, how does the time (number of steps, comparisons, etc…) grow?

Content Copyright Nanyang Technological University

Application: Expression evaluation

- Almost all higher-level programming languages let you evaluate complex expressions, such as **3*x+y+4*z**
- You may occasionally need to evaluate expressions in your own program, without benefit of a compiler
- Our typical notation where operands appear on the left and right sides of an operator, is known as "**infix notation**"
- In contrast, "**postfix notation**" places the two operands to the left of the operator.
- For example:
  - **Infix notation: 3 * x + y + 4 * z**
  - **Postfix (RPN) notation: 3 x * y + 4 z * +**

Content Copyright Nanyang Technological University

Performing calculations

- To evaluate any expression, such as 1+2*3+4 we make two passes through the expression:
  - Convert the “infix notation” expression to “postfix notation” (also known as “Reverse Polish Notation”, RPN)
  - Evaluate the postfix expression to compute its value
  - Each pass uses a stack

Content Copyright Nanyang Technological University

How to convert Infix to Postfix

Input	Action
Operand	Write to output
Operator	- While operator on stack top is >= precedence of the input operator – pop and output the operator from stack top. - Push the input operator onto stack
<END>	While stack not empty, pop and output the operator at top of stack

Content Copyright Nanyang Technological University

Converting 1+2*3+4 to postfix

Input token	Action	Output so far	Stack (bottom --> top)
1	Emit “1” to output	1	
+	Push “+” on stack	1	+
2	Emit “2” to output	1 2	+
*	Push “*” on stack since it has higher precedence than current top of stack	1 2	+ *
3	Emit “3” to output	1 2 3	+ *
+	Pop & emit all operators from stack that are >= in precedence. Then push “+” on stack.	1 2 3 * +	+
4	Emit “4” to output	1 2 3 * + 4	+
<END>	Pop & emit remaining operators	1 2 3 * + 4 +	

Output:

1 2 3 * + 4 +

Content Copyright Nanyang Technological University

Infix to Postfix (simplified version)

```
# return the precedence of the operator
# 1 for + or -
# 2 for * or /
def precedence(operator):
    if operator in "+-":
        return 1
    else:
        return 2

# Parse off the next token from the string.
# A token is an operator or an operand.
# Return the token, and the shrunken string
def nextToken(s):
    token = ""
    if len(s) > 0:
        # its an operator, so take just this character
        if s[0] in "+-*/":
            token = s[0]
            s = s[1:]

        # its an operand, so take all its characters
        else:
            while len(s) > 0 and s[0] not in "+-*/":
                token += s[0]
                s = s[1:]

    return token, s
```

Content Copyright Nanyang Technological University

Infix to Postfix (part 2 – simplified version)

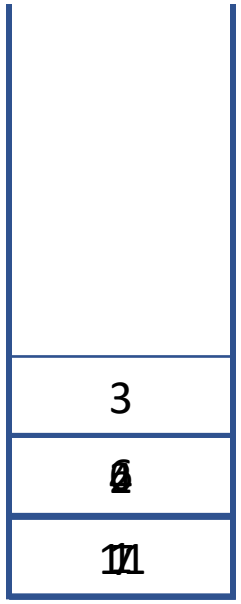
```
# Return the postfix version of the input formula
# in the form of a Python list of tokens
def toPostfix(formula):
    ans = []
    s = Stack(1000)

    # for each token in the formula  (fencepost loop)
    token, formula = nextToken(formula)
    while token:

        # if it's an operator, while the operator on the stack top
        # is >= precedence of this operator token
        # pop & output the operator at the stack top
        if token in "+-*/":
            while s.peek() and precedence(s.peek()) >= precedence(token):
                ans += [s.pop()]

            # and then push this operator token onto the stack
            s.push(token)

        else: ans += [token]  # operand tokens go straight to output

        token, formula = nextToken(formula)

    # pop the remaining operators and append to answer
    while not s.isEmpty(): ans += [s.pop()]

    return ans
```

Content Copyright Nanyang Technological University

Evaluating a Postfix expression
Infix: 1+2*3+4 Postfix: 1 2 3 * + 4 +

Input	Action
Operator + - * /	- Pop two operands from stack - Combine them depending upon the operator - Push result back onto stack
Operand	Push int() of operand to stack
End (empty input list)	Pop final result from stack

Input	Stack
“1”	1
“2”	1 2
“3”	1 2 3
“*”	1 6
“+”	7
“4”	7 4
“+”	11



Output:

11

Content Copyright Nanyang Technological University

Evaluate a Postfix expression

```
# takes as input a list of tokens defining an expression in postfix order.
# evalutes the expressions and returns the resulting value.
def processPostfix(postfixTokens):
    s = Stack(1000)
    for t in postfixTokens:                     # for each token
        if t in "+-*/":                         # if its an operator...
            op2 = s.pop()                       # pop the two operands
            op1 = s.pop()                       # 2nd operand was on top of stack
            if t == '+':   ans = op1 + op2      # combine the two operands
            elif t == '*': ans = op1 * op2
            elif t == '-': ans = op1 - op2
            elif t == '/': ans = op1 / op2
            s.push(ans)                         # push the resulting value

        else:                                   # not an operator
            s.push(int(t))                      # so push the int() of the operand

    return s.pop()                              # remaining value on stack
                                                # must be value of expression
```

Content Copyright Nanyang Technological University

Comparing algorithm use of stacks

- Conversion of Infix to Postfix: uses a stack of strings to keep track of the ***operators*** still to be processed.

- Evaluation of a Postfix expression: uses a stack of ints that correspond to the numbers/***operands*** still to be combined

Content Copyright Nanyang Technological University

Summary

- Stacks are useful for working with nested structures, such as:
  - Arithmetic expressions
  - Nested statements in a programming language

Content Copyright Nanyang Technological University